

- Description

This module manages a smart library book loan and availability monitoring system, ensuring that:

1. **Book availability is continuously monitored to prevent overbooking and optimize loan scheduling.**
2. **If a book is fully borrowed, users are added to a waiting list and notified when it becomes available.**
3. **If multiple copies are available, the system distributes requests efficiently to balance usage.**
4. **A log of all loan transactions, return events, and waiting list updates is maintained for efficient library management.**

This system helps improve book accessibility, reduce waiting times, and optimize library resource allocation.

- Specification

module SmartLibraryLoanManagement

var availableCopies, totalCopies: integer

var waitlist: sequence of string

var log: sequence of string

process borrowBook(user: string)

functional scenario form:

availableCopies > 0 && availableCopies = availableCopies - 1, log = log $\oplus$ (user + " borrowed a book | Remaining: " + availableCopies + "/" + totalCopies), Book borrowed successfully.

||

availableCopies = 0 && waitlist = waitlist $\oplus$ user, log = log $\oplus$ (user + " added to waitlist | No copies available."), User added to waitlist.

end process

process returnBook(user: string)

functional scenario form:

(size(waitlist) > 0) && let nextUser = head(waitlist) in waitlist = tail(waitlist), log = log $\oplus$ (user + " returned a book | Assigned to " + nextUser), Book assigned to next user in waitlist.

OR

(size(waitlist) = 0) && availableCopies = min(totalCopies, availableCopies + 1), log = log $\oplus$ (user + " returned a book | Available: " + availableCopies + "/" + totalCopies), Book returned to available copies.

end process

end module

- Java Code

```
package code;
```

```

import java.util.*;
class Book {
    private String title;
    private int totalCopies;
    private int availableCopies;
    private Queue<String> waitlist;
    private List<String> log;
    public Book(String title, int totalCopies) {
        this.title = title;
        this.totalCopies = totalCopies;
        this.availableCopies = totalCopies;
        this.waitlist = new LinkedList<>();
        this.log = new ArrayList<>();
    }
    public void borrowBook(String user) {
        if (availableCopies > 0) {
            availableCopies--;
            System.out.printf("%s borrowed '%s' | Available Copies: %d/%d\n", user, title, availableCopies, totalCopies);
            log.add(String.format("%s borrowed | Remaining: %d/%d", user, availableCopies, totalCopies));
        } else {
            waitlist.add(user);
            System.out.printf("No copies available. %s added to the waiting list for '%s'.\n", user, title);
            log.add(String.format("%s added to waitlist | No copies available", user));
        }
    }
    public void returnBook(String user) {
        if (!waitlist.isEmpty()) {
            String nextUser = waitlist.poll();
            System.out.printf("%s returned '%s'. %s can now borrow it.\n", user, title, nextUser);
            log.add(String.format("%s returned | Assigned to %s", user, nextUser));
        } else {
            availableCopies = Math.min(totalCopies, availableCopies + 1);
            System.out.printf("%s returned '%s' | Available Copies: %d/%d\n", user, title, availableCopies, totalCopies);
            log.add(String.format("%s returned | Available: %d/%d", user, availableCopies, totalCopies));
        }
    }
}

```

```

    }
    public void printLog() {
        System.out.printf("\n'%s' Loan Log:%n", title);
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

public class c59_SmartLibrarySystem {
    public static void main(String[] args) {
        Book book1 = new Book("Machine Learning Basics", 2);
        // testcase-VT:
        //     book1.borrowBook("Alice");
        //     book1.borrowBook("Bob");
        //     book1.borrowBook("Charlie");
        //     book1.returnBook("Alice");
        //     book1.returnBook("Bob");
        //     book1.printLog();

        // testcase-FT:
        //     book1.borrowBook("");
        //     book1.borrowBook("o");
        //     book1.borrowBook("A");
        //     book1.borrowBook("??");
        //     book1.borrowBook("G4u");
        //     book1.printLog();

        // testcase-Z3:
        //     book1.borrowBook("Alice");
        //     book1.borrowBook("Bob");
        //     book1.borrowBook("Charlie");
        //     book1.returnBook("Alice");
        //     book1.returnBook("Bob");
        //     book1.printLog();

        // testcase-UVT:
        book1.borrowBook("Alice");
        book1.borrowBook("Bob");
        book1.borrowBook("Charlie");
        book1.returnBook("Alice");
        book1.returnBook("Bob");
        book1.printLog();
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c59_SmartLibrarySystem.java	100.0 %	6	0	6
> Book	100.0 %	6	0	6
> c59_SmartLibrarySystem		0	0	0

3. Vibration testing instruction coverage: 98.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	273	20,392	20,665
▼ src	1.3 %	273	20,392	20,665
▼ code	1.3 %	273	20,392	20,665
▼ c59_SmartLibrarySystem.java	98.9 %	273	3	276
> Book	100.0 %	249	0	249
> c59_SmartLibrarySystem	88.9 %	24	3	27

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c59_SmartLibrarySystem.java	66.7 %	4	2	6
> Book	66.7 %	4	2	6
> c59_SmartLibrarySystem		0	0	0

5. Fuzz testing instruction coverage: 60.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	167	20,498	20,665
▼ src	0.8 %	167	20,498	20,665
▼ code	0.8 %	167	20,498	20,665
▼ c59_SmartLibrarySystem.java	60.5 %	167	109	276
> Book	57.4 %	143	106	249
> c59_SmartLibrarySystem	88.9 %	24	3	27

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c59_SmartLibrarySystem.java	100.0 %	6	0	6
> Book	100.0 %	6	0	6
> c59_SmartLibrarySystem		0	0	0

7. Z3 instruction coverage: 98.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	273	20,392	20,665
▼ src	1.3 %	273	20,392	20,665
▼ code	1.3 %	273	20,392	20,665
▼ c59_SmartLibrarySystem.java	98.9 %	273	3	276
> Book	100.0 %	249	0	249
> c59_SmartLibrarySystem	88.9 %	24	3	27

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c59_SmartLibrarySystem.java	100.0 %	6	0	6
> Book	100.0 %	6	0	6
> c59_SmartLibrarySystem		0	0	0

9. Unimproved V-method instruction coverage: 98.9%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓  VibrationTestingTool		 1.3 %	273	20,392	20,665
✓  src		 1.3 %	273	20,392	20,665
✓  code		 1.3 %	273	20,392	20,665
✓  c59_SmartLibrarySystem.java		 98.9 %	273	3	276
>  Book		 100.0 %	249	0	249
>  c59_SmartLibrarySystem		 88.9 %	24	3	27